Our Research Paper Explained

Every section, word, symbol, table, figure and number of the paper, in simple language

Mihir Bhati (24BCE5341) · Ahana Sharan (24BCE5334) · Guide: Vijayaprabakaran K

VIT Chennai · Companion to paper/main.tex (8 pages, IEEE conference format)

How to use this document. Keep the paper open next to it. Sections 3 to 12 follow the paper in order: for each part of the paper you get what it says, what every word in it means, and why it is there. Section 2 is a dictionary of every technical word. Section 13 says where every number comes from. Section 14 has the questions a teacher or reviewer is likely to ask, with honest answers.

Contents

1. The whole paper in one minute
2. Dictionary: every technical word in the paper
3. The title, word by word
4. The author block
5. The abstract, sentence by sentence
6. The keywords
7. Section I: Introduction
8. Section II: Literature survey (all 15 papers) and Table I
9. Section III: Proposed methodology, with every symbol and equation
10. Section IV-A to IV-D: Data, model, baselines, statistics
11. Section IV-E to IV-K: Results, every table and figure
12. Section V: Conclusion, acknowledgment and references
13. Where every number in the paper comes from
14. Questions a teacher or reviewer may ask, and the answers

1. The whole paper in one minute

A voice or chat assistant has to turn a sentence like "wake me up at seven tomorrow" into two things a program can use: the **intent** (what the user wants: `alarm_set`) and the **slots** or **entities** (the details: time = "seven", date = "tomorrow"). It must also say "I can't do that" to requests it was not built for.

A small AI model (Qwen2.5, 3 billion parameters) can do this from a well-written prompt, without training. But someone has to decide **how** to write that prompt: one question or two, with or without label descriptions, which examples to show, and who decides that a request is out of scope. We call each combination of these choices a **design**. There are 12 designs, and the best and worst differ by 13 to 16 points of accuracy.

Our paper builds an **agent** that makes these choices automatically. It reads the project description, tries 4 of the 12 designs on 100 practice requests, picks the best by a fixed rule and saves it as a package. We then test it fairly on requests it has never seen, against a generic prompt, a human, random choice and a trained XLM-R model.

The main results in four lines.
1. The improved agent (A2) found the best of the 12 designs on all three projects while trying only 4.
2. On the two development projects (P1, P2) it beat the generic prompt and met our pre-set success rule.
3. On the held-out project (P3) its intent score was 1.3 points lower (not a real difference statistically), but its entity score was 14.6 points higher.
4. It beat XLM-R trained on the same small data on every project.

2. Dictionary: every technical word in the paper

Words are grouped by topic. Each one is explained the way you would explain it to a friend.

2.1 The task

Word	Meaning in simple words	Example from our project
NLU (natural language understanding)	Getting a computer to understand what a sentence means, not just read it.	Understanding that "turn off the lights" means switch the lights off.
Task-oriented assistant	An assistant built to do specific jobs (set alarms, send emails), not to chat freely.	A smart-home app.
Intent	The type of request: what the user wants done. Exactly one per sentence.	`alarm_set`, `iot_hue_lightoff`, `email_sendemail`.
Intent inventory (I)	The full list of intents one app supports.	P1 has 12 intents.
Slot / entity	A detail inside the sentence that the app needs. Both words mean the same thing here.	time = "seven", date = "tomorrow".
Entity type	The kind of detail.	`time`, `date`, `place_name`, `person`.
Entity schema (E)	The list of entity types an app uses, and which intent may carry which type.	`alarm_set` may carry `time` and `date`.
Span	The exact piece of the sentence that is the entity, given by start and end character positions.	"seven" starts at character 14 and ends at 19 in "wake me up at seven tomorrow".
Slot filling	Finding the entities in the sentence.	Finding "seven" and labelling it `time`.
Out-of-scope (OOS)	A request the app was not built for. The right answer is to refuse.	Asking the smart-home app "what's the weather?".
Rejection	The app saying "not supported" instead of guessing an intent.	Answer `none_of_the_above`.
Pipeline	The full chain from the user's sentence to the final answer: prompt, model, checks, output.	
Project	One app with its own intents and entities. We built three: P1, P2, P3.	P1 = home assistant (alarms + smart devices).

2.2 The models

Word	Meaning in simple words
LM / language model	An AI that reads text and writes the most likely continuation. "LLM" = large language model.
Instruction-tuned	A model trained to follow instructions ("Reply with JSON only") rather than just continue text.
Parameters	The numbers inside a model that store what it learned. 3 billion parameters = a small modern model.
Qwen2.5-3B-Instruct	The language model we use for everything: made by Alibaba's Qwen team, 3 billion parameters, instruction-tuned, free to run locally. Reference [17].
Quantised to 4 bits (Q4_K_M)	Each parameter is stored in about 4 bits instead of 16, so the model fits in 4 GB of laptop GPU memory. Slightly less precise, much smaller and faster.
Ollama	The free program that runs the model on our laptop and answers requests at 127.0.0.1:11434.
Temperature 0, seed 0	Settings that make the model always pick its single most likely answer, so the same prompt always gives the same reply (we checked: 50 of 50 identical).
Token	A piece of a word the model reads. Roughly 4 characters. Our prompts average 1,028 tokens.
Prompt	The text we send to the model: instructions, label list, examples, then the user's sentence.
Demonstration / example / shot	A solved example inside the prompt: a sentence with its correct answer.
k-shot	Having k labelled examples per intent. We use k = 10.
In-context learning	The model learning a task just from examples in the prompt, without any training.
Fine-tuning / training	Changing a model's parameters by showing it many labelled examples.
Encoder	A model that turns a sentence into numbers for classification (does not write text). BERT and XLM-R are encoders.
XLM-R (base)	A well-known encoder from Facebook AI, trained on 100 languages [5]. We fine-tune it as baseline B3.
Softmax probability	The confidence an encoder gives each intent, summing to 1. Low top confidence suggests out-of-scope.

2.3 Our system's parts

Word	Meaning in simple words
Agent	A program that uses a language model to make decisions and act on the results. Ours designs pipelines.
Design (θ)	One way of building the pipeline: one choice for each of the 3 factors. 12 designs in total.
Design space (Θ)	The set of all 12 designs.
Factor	One decision: extraction strategy (X), example selection (S), rejection policy (O).
Candidate	A design the agent decides to test.
Budget (B = 4)	The agent may test at most 4 designs: 3 first, then 1 revision.
Joint	Extraction strategy: one call returns intent and entities together.
Two-stage	Extraction strategy: first call finds the intent, second call finds only that intent's entities.
Schema-guided	Extraction strategy: one call, but the prompt includes a one-line description of every intent and entity type.
Fixed examples	Example selection: the same one example per intent in every prompt.
Similarity examples	Example selection: the 16 pool examples most similar to the user's sentence.
Model label (llm_label)	Rejection policy: the model may answer `none_of_the_above`.
Threshold (tfidf_threshold)	Rejection policy: refuse before calling the model if the sentence is not similar enough to any example.
Design names	Written extraction-examples-rejection, e.g. `schema-sim-llm` = schema-guided, similarity examples, model label.
A1	The first agent (direct planner): asks the model to name 3 designs directly.
A2	The improved agent (factor-wise planner): asks the model to rank options per factor, then builds candidates in a fixed pattern.
Revision	After testing 3 candidates, the agent picks one more design to try based on the results.
Error report	The summary the agent shows the model during revision: each candidate's scores and error rates.
Selection rule	The fixed rule that picks the winner: highest intent F1, then entity F1, then speed.
Package / export	The saved final pipeline: design, descriptions, examples, threshold, prompts, model identity and file hashes.
Hash (SHA-256)	A fingerprint of a file. If even one character changes, the fingerprint changes. Used to prove nothing was altered.
Model digest	The fingerprint of the model's weights (ours: 357c53fb659c…). Proves the same model is used.
Schema / contract	The required format of the model's reply: an intent from the list and entities with allowed types.
Schema validity	Fraction of replies that follow that format.
Repair	If a reply breaks the format, the model gets one more try with the exact error explained.
Span-location rule	How we turn the entity text the model returns into character positions. Exact match only.
Unsupported entity	An entity text the model returned that does not appear in the sentence (an invented value).

2.4 Data and evaluation

Word	Meaning in simple words
MASSIVE	A public dataset by Amazon of assistant requests labelled with intents and slots [1]. We use the English part (16,521 sentences).
Scenario / domain	A topic group in MASSIVE, e.g. alarm, iot (smart devices), calendar, email.
Pool (K)	The few labelled examples the agent may use: 10 per intent.
Development set (V, dev)	100 labelled practice requests used to choose designs and thresholds (80 in-scope + 20 OOS).
Test set	Requests locked away until everything was decided; used once for the final, honest score.
Held-out project (P3)	A whole project kept locked during development, to check the method works on something new.
Freeze / preregistration	Writing down and locking all choices, rules and code in one commit before looking at test data.
Precision	Of the answers the system gave, how many were right.
Recall	Of the right answers that existed, how many the system found.
F1	One number combining precision and recall (their harmonic mean). 1.0 is perfect, 0 is useless.
Macro-F1	Compute F1 for each intent separately, then average, so rare intents count as much as common ones.
Micro-F1	Count all hits and misses together first, then compute one F1. Used for entities.
Intent macro-F1	Our main score for intents.
Entity span F1	Our score for entities: a hit only if type, start and end all match exactly.
OOS F1	How well the system refuses out-of-scope requests (refusal treated as the thing to find).
Point	One hundredth of an F1 score. 0.898 − 0.876 = 0.022 = 2.2 points.
Latency	How long one request takes, in seconds.
Median	The middle value: half the requests are faster.
p95 (95th percentile)	95 of every 100 requests are faster than this. Shows the slow cases.
Baseline	A simpler system we compare against, to show whether ours is better.
Ablation	Removing one part of the system to see how much that part matters.
Random search	Trying designs chosen at random. The fair test of whether an agent's choices are smart.
Paired bootstrap	A way to check if a difference is real: resample the test set 1,000 times and see how the difference varies.
95% confidence interval (CI)	The range the true difference probably lies in. If it does not include 0, the difference is significant.
Significant	Unlikely to be luck. In our paper: the 95% interval excludes zero.
n.s.	Not significant.
Lexicographic	Compare by the first criterion; only on a tie use the second; and so on (like sorting words in a dictionary).

3. The title, word by word

"Automated Design of Task-Specific Intent and Slot Pipelines with a Small Language-Model Agent: A Controlled Evaluation on MASSIVE"

Part of the title	What it tells the reader
Automated Design	The system makes the design decisions, not a person.
Task-Specific	Each app (task) gets its own pipeline, built for its own intents.
Intent and Slot Pipelines	What is designed: the chain that finds intents and slots.
Small Language-Model Agent	The designer is an agent using a small (3B) model that runs on a laptop, not a giant cloud model.
A Controlled Evaluation	The testing is fair and strict: fixed rules, frozen before testing, same data and scorer for everyone.
on MASSIVE	The dataset used.

4. The author block

- Three authors: Mihir Bhati and Ahana Sharan (students, marked * for **equal contribution**, explained in the footnote) and Vijayaprabakaran K (guide).
- Affiliation: School of Computer Science and Engineering, Vellore Institute of Technology, Chennai.
- Registration numbers and emails are given so readers can contact the authors.

5. The abstract, sentence by sentence

The abstract is 198 words (the guideline is 150 to 200). It is the only part many readers see, so every sentence carries a fact.

Sentence (shortened)	What it means
Building the intent and slot component still requires a developer to decide, by trial and error, how the model is prompted, which examples it sees and how it rejects unsupported requests.	The problem: these are manual design choices today.
We present an agent that makes them from a project specification: description, intent inventory, entity schema, ten labelled utterances per intent.	Our solution and its input. "Utterance" = one user sentence.
It validates and enriches the specification, ranks options, evaluates four of twelve candidates on 100 development utterances, revises one factor, selects under constraints, and exports a hash-verified package.	The six phases of the method, in order (Section III).
Every planning and runtime call uses one 3-billion-parameter model on a laptop.	No cloud and no big model: the same small model plans and answers.
On three projects built from MASSIVE, one held out until the protocol was frozen, the agent chose the development-optimal design every time.	"Development-optimal" = the design that scores best of all 12 on the practice set.
Against a generic prompt it raised intent F1 by 2.2 and 4.9 points and entity F1 by 7.3 and 5.5 points on the two development projects, meeting the preregistered success rule.	The main win, on P1 and P2. "Preregistered" = the rule was fixed before testing.
On the held-out project intent F1 fell by 1.3 points (not significant) while entity F1 rose by 14.6.	Honest report of P3: slightly worse on intents (could be luck), much better on entities.
It also scored above a k-shot XLM-R baseline on all three projects.	Beats a trained encoder given the same small data.

6. The keywords

IEEE asks for keywords ("Index Terms") so the paper can be found by search. Ours name the task (natural language understanding, intent classification, slot filling, out-of-scope detection), the method (language model agents, in-context learning) and the contribution (automated pipeline design).

7. Section I: Introduction

The introduction has five parts. Each one answers a question a reader has.

Paragraph 1: What is the problem domain?

It defines the task with the alarm example, says every app has its own intents (smart home, calendar, food ordering share almost none), and says trained encoders [1], [2] work well only with thousands of labelled sentences. The number 1,159 is how many training sentences MASSIVE has for alarm + iot alone: a new app rarely has that many.

Paragraph 2: Why is it still hard with language models?

Small instruction-tuned models remove training, run on a 4 GB laptop GPU and answer in under half a second (our measured median is 0.42 s). But the four design decisions remain. The key evidence: with model and data fixed, the 12 designs range from 0.839 to 0.974 intent F1 on P1 (13.5 points) and 0.738 to 0.898 on P2 (16.0 points). So the choice matters a lot.

Paragraph 3: What have others done, and what is missing?

Others automated rewriting the instruction text [4]–[6], tuning examples inside a fixed program [7], or replacing the model with a trained small model [8]. None produces intent + entity spans + rejection under deployment limits, and they are rarely compared with random search [9]. Without that comparison you cannot tell if the AI's reasoning helped or if trying several options at random would do just as well.

Paragraph 4: What exactly does this paper ask?

Can an agent with only 4 tries design a pipeline that beats a generic prompt, random search, a human and a fine-tuned encoder on the same examples?

The three contributions

1. **Formulation and agent:** we describe pipeline design as "pick the best option from a fixed list under rules" and build an agent that does the full job from analysis to a verified package.
2. **Factor-wise planning:** building candidates one factor at a time and revising exactly one factor found the best design every time, even though the model's own first preferences were wrong on two factors.
3. **Controlled evaluation:** three projects (one held out), six kinds of baselines, statistics with confidence intervals.

8. Section II: Literature survey (all 15 papers) and Table I

Our teacher asked for about 15 relevant papers, a short paragraph each, and one believable research gap. Each paragraph says what the paper did and, in its last sentence, how it connects to our work. The citation number [n] is the paper's number in the reference list.

[3] FitzGerald et al., MASSIVE (ACL 2023). Released one million assistant requests in 51 languages, translated from the English SLURP corpus [10], with 60 intents, 18 scenarios and 55 slot types. Their baselines (XLM-R, mT5) train on the full data. **Link to us:** Our dataset. We take only the English part and group a few scenarios into each project to get small, realistic apps.

[11] Larson et al., CLINC150 (EMNLP 2019). A dataset of 150 intents plus out-of-scope queries. Showed that models accurate on normal requests still miss most out-of-scope ones, whether they use a confidence threshold or an extra "out-of-scope" class. **Link to us:** Why rejection is its own factor in our design space and why we report OOS F1 separately.

[1] Chen et al., BERT for joint intent and slot (2019). Used one BERT model with two outputs: the intent from the sentence summary, the slot labels from each word. Training both together beat earlier models on ATIS and Snips. **Link to us:** The standard trained approach; our XLM-R baseline uses the same two-output design.

[2] Conneau et al., XLM-R (ACL 2020). Trained a large encoder on 2.5 TB of web text in 100 languages; strong results on classification, question answering and named entities. **Link to us:** We fine-tune its base version as baseline B3, on 10 per intent (B3-k) and on the full training data (B3-full).

[12] Brown et al., GPT-3 (NeurIPS 2020). Showed a big language model can do a new task from instructions and a few examples in the prompt, with no training; more examples and bigger models do better. **Link to us:** The idea our runtime rests on. Once the prompt is the program, what goes into it becomes a design decision.

[13] Liu et al., good in-context examples (DeeLIO 2022). Picking the examples most similar to the question works better than random examples. **Link to us:** Our "similarity examples" option does the same, with a cheap TF-IDF index instead of a second model.

[4] Zhou et al., APE (ICLR 2023). An LM writes many candidate instructions, each is scored, the best is kept. Often as good as human-written. **Link to us:** Searches free text for single-label tasks; we search a structured design space for a full pipeline.

[5] Pryzant et al., ProTeGi (EMNLP 2023). Describes a prompt's mistakes in words (a "gradient"), edits the prompt to fix them, keeps the best edits. **Link to us:** Our revision step also shows the model an error report, but only allows one move in a fixed list, so a bad suggestion wastes one test and can never produce a broken pipeline.

[6] Yang et al., OPRO (ICLR 2024). Shows the LM past solutions with their scores and asks for a better one, over many rounds. **Link to us:** OPRO needs many rounds; we can afford one revision because each test costs about 100 model calls.

[7] Khattab et al., DSPy (ICLR 2024). Write an LM pipeline as modules; a "compiler" automatically finds good examples and instructions. **Link to us:** DSPy tunes what is inside a structure the developer wrote;

our agent chooses the structure itself.

[8] Viswanathan et al., Prompt2Model (EMNLP 2023 demo). From a task description, finds datasets, generates more data and trains a small model to deploy. **Link to us:** Closest in aim (description → deployable system) but needs training and was not tested on slots or rejection.

[14] Madaan et al., Self-Refine (NeurIPS 2023). The same LM writes, criticises and rewrites its own output repeatedly. **Link to us:** We use a small form at runtime: one repair attempt when a reply breaks the format.

[15] Yao et al., ReAct (ICLR 2023). The LM alternates thinking and acting (e.g. searching), using what it observes. **Link to us:** Our agent follows the same think → act (test) → observe (scores) → act again pattern.

[9] Bergstra and Bengio, random search (JMLR 2012). Random search finds good settings faster than trying a full grid, because only a few settings matter. **Link to us:** Why random search (B4) is the fair test of whether our agent's choices are smart.

[16] He et al., AutoML survey (KBS 2021). Reviews automated machine learning: searching over data steps, features, settings and network designs. **Link to us:** In AutoML terms our system has a small hand-made search space, an LM as the search method and dev-set testing as the costly step.

Three further references are cited but are not part of the 15: SLURP [10] (where MASSIVE's English text comes from), the Qwen2.5 report [17] (our model) and Koehn [18] (the bootstrap test).

The research gap (Section II-G)

> **Gap, in one line:** nobody has tested whether a small local LM agent, given only a project description and a fixed budget, picks better complete intent + slot pipelines than random choice, a generic prompt and a human, under a frozen protocol.
> **Why it is believable for a student project:** it needs no new model or big compute, only careful design and fair testing, which is what we did.

Table I: comparison with related work, column by column

Column	What it asks of each work
What is searched or learned	What the method changes: model weights, instruction text, examples, or (ours) the pipeline design.
Labelled data needed	How much labelled data it needs. Ours: 10 per intent plus 100 dev sentences.
Slot spans	Does it output entities with positions? Only the trained encoders and ours do.
OOS rejection	Does it handle out-of-scope requests? Only ours.
Deployable artefact	What you get at the end: a model, a prompt, a program, or (ours) a hashed package.
Random-search control	Was it compared with random choice? Liu et al. compared with random examples; Pryzant et al. with Monte-Carlo sampling; ours with 5 seeds of random search over the same 12 designs.

9. Section III: Proposed methodology, with every symbol and equation

This section describes the method without naming any software (teacher's rule). Figure 1 shows it.

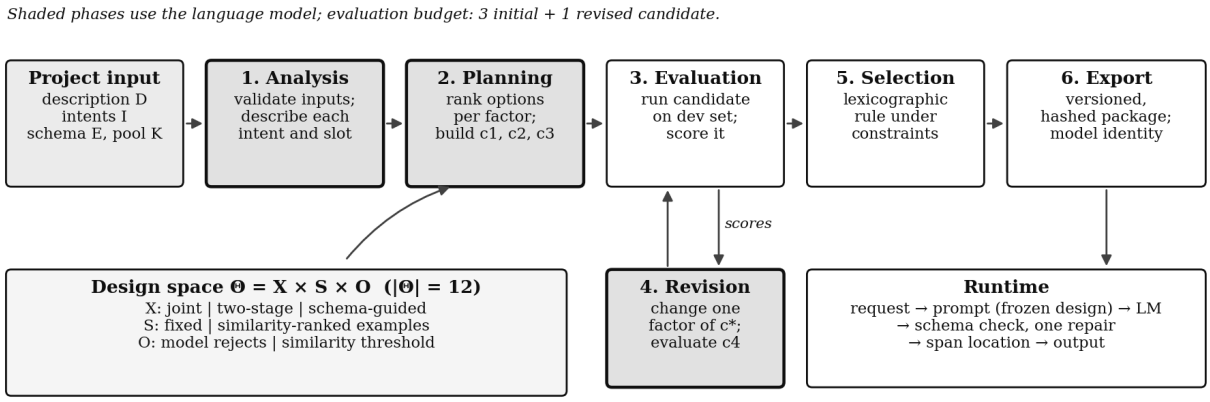


Figure 1 of the paper: the six phases. Grey boxes use the language model.

III-A Problem formulation: the symbols

Symbol	Name	Meaning	P1 example
P = (D, I, E, K, C)	Project	Everything the agent is given about one app.	The home assistant.
D	Description	Free text describing the product.	"A home assistant that sets alarms and controls lights…"
I	Intent inventory	The list of intents.	12 intents.
E	Entity schema	Entity types and which intent may carry which.	18 entity types.
K	Pool	10 labelled sentences per intent.	120 sentences.
C	Constraints	Limits the result must respect: S_min, a minimum entity F1, L_max.	Validity ≥ 0.98, p95 ≤ 1.22 s.
V	Development set	Labelled practice requests, including out-of-scope ones.	100 (80 + 20).
u	Utterance	One user sentence.	"wake me up at seven tomorrow"
(î, Ŝ)	Prediction	Predicted intent and predicted set of spans (the hat ^ means "predicted").	(alarm_set, {time 14–19, date 20–28})
θ, Θ	Design, design space	One design; the set of all 12.	schema-sim-llm
B = 4	Budget	Maximum designs tested.	3 + 1

III-B Design space: the 3 factors and equations (1) and (2)

Θ = X × S × O means "every combination of one X, one S and one O": 3 × 2 × 2 = 12 designs. The agent only chooses among these trusted components; it never writes code, so it cannot produce something that does not run.

Equation (1), cosine similarity: sim(u, x) compares the user's sentence u with a pool sentence x. Both are turned into TF-IDF vectors ϕ (a list of numbers, one per word or word pair, where rare words get more weight). The cosine is 1 when they use the same words in the same proportions and 0 when they share no words. "Sublinear term frequency" means a word repeated many times does not count proportionally more.

Equation (2), the threshold τ*: for the threshold rejection policy, a request is refused if its highest similarity to any pool sentence is below τ*. We try every similarity value seen on the dev set as a possible τ, and keep the one with the best macro-F1 over all intents plus the OOS class. On a tie we keep the lower threshold, which refuses fewer real requests. The star * means "the best value**

Prompt limit: at most 16 examples and 2,000 input tokens. If too long, examples are removed first; label definitions are never cut.

III-C to III-I: the six phases

Phase	What happens	Uses the LM?
1. Analysis	Validation: checks for duplicate names, examples with unknown labels, intents with fewer than 10 examples, entity types never seen in the pool. Enrichment: 2 LM calls write a one-line description for every intent and entity type. The LM may only describe existing names, never add or rename.	Yes (2 calls)
2. Planning	A1: the LM names 3 designs. A2: the LM ranks the options of each factor separately, and the agent builds c1 = all top choices, c2 = other example selection, c3 = runner-up extraction (equation 3).	Yes (1 call)
3. Evaluation	Each candidate runs on all 100 dev requests and is scored. An error report is written.	Yes (≈100 calls per candidate)
4. Revision	From the best so far (ĉ), the LM picks one move from N(ĉ): untried designs that differ in exactly one factor (equation 4). That design is the 4th and last test.	Yes (1 call)
5. Selection	Remove candidates breaking a constraint (equation 5), then pick by the lexicographic rule (equation 6).	No
6. Export	Save the package with SHA-256 hashes of every file and the model digest. Loading refuses if anything changed.	No

Equation (3) is just the list c1 = (x1, s1, o1), c2 = (x1, s2, o1), c3 = (x2, s1, o1), where x1 is the model's favourite extraction, x2 its second, and so on. Why this pattern: whatever the model prefers, the other example option and the second-best extraction are always tested once. This is what saved A2 when the model's preferences were wrong (Section 11).

Equation (4), the neighbourhood N(ĉ): "all designs not yet tested (Θ minus $\mathscr{E}$) that differ from the best so far in exactly one factor." $\mathscr{E}$ is the set already evaluated.

Equation (5), eligible set: keep candidates with schema validity SV ≥ S_min, entity F1 ≥ f_min, and estimated p95 latency ≤ L_max.

Equation (6), the selection rule: c* = the eligible candidate with the highest intent macro-F1; if tied, highest entity F1; if still tied, lowest latency estimate; finally the design name, so the result is always the same. "lexmax" = lexicographic maximum. No weighted average is used, so a trade-off cannot be hidden. If no candidate is eligible, the agent says so instead of relaxing the rules. Random search (B4) uses this exact rule.

> **Example of the selection rule (A2 on P1, dev scores):** schema-sim-llm 0.974 / 0.707, schema-sim-tfidf 0.869 / 0.617, schema-fixed-tfidf 0.845 / 0.505, twostage-fixed-tfidf 0.839 / 0.544. All are eligible; the highest intent F1 is 0.974, so schema-sim-llm wins. No tie-break was needed.

Runtime: what happens to one request

1. Under the threshold policy, refuse immediately if the sentence is too unlike the pool.
2. Build the prompt from the frozen design and send it to the model (temperature 0, JSON output).
3. Check the reply: is it an object, is the intent in the list (or none_of_the_above), are the entity types allowed?
4. If not, one repair call explaining the exact error. If it fails again, it counts as invalid and stays in every score.
5. Turn each entity's text into character positions with the span-location rule.

The span-location rule, with an example

The model returns entity **text**, like "seven", not positions. The rule: strip spaces and quotes from the ends; look for the text in the sentence exactly (case-sensitive), then ignoring case; if it appears more than once, take the first place not already used by another entity. If it is not in the sentence at all, it is **unsupported** and counts as a wrong answer. We never "fuzzy-match" (accept near spellings), and every LM system uses this same rule, so none gets an unfair advantage.

> Sentence: "wake me up at seven tomorrow". Model returns time = "seven" → found at characters 14–19 ✓. Model returns date = "tmrw" → not in the sentence → unsupported, counts as a false positive.

III-J Evaluation metrics: equations (7) and (8), with worked examples

Equation (7), intent macro-F1: for each intent i, precision P_i = correct predictions of i ÷ all predictions of i; recall R_i = correct predictions of i ÷ all true i; F1_i = 2·P·R ÷ (P + R). Then average over the |I| intents. Refusing a real request, or an invalid reply, counts as a miss for its true intent.

> **Worked example.** 4 requests: 3 are truly alarm_set, 1 is alarm_query. The system answers alarm_set, alarm_set, alarm_query, alarm_query.
> alarm_set: predicted twice, both right → P = 2/2 = 1.0; 3 true, 2 found → R = 2/3 = 0.667; F1 = 0.80.
> alarm_query: predicted twice, 1 right → P = 0.5; 1 true, found → R = 1.0; F1 = 0.667.
> Macro-F1 = (0.80 + 0.667) / 2 = **0.733**.

Equation (8), entity span F1: G = the true spans, $\hat{S}$ = predicted spans. A hit (G ∩ $\hat{S}$) needs the same type, start and end. P^E = hits ÷ predicted; R^E = hits ÷ true; F1 as usual. Counted over all in-scope requests together (micro-average).

Worked example. True: time "seven" (14–19), date "tomorrow" (20–28). Predicted: time "seven" (14–19), time "tomorrow" (20–28, wrong type).
Hits = 1. P = 1/2 = 0.5, R = 1/2 = 0.5, F1 = **0.5**. The wrong type makes the second one a miss even though the text is right.

OOS F1: the same F1 formula where "refused" is the thing being found, over all requests. **Schema validity SV:** fraction of final replies that follow the format. **Latency:** median and p95 of seconds per request, only from runs with the cache turned off.

10. Section IV-A to IV-D: data, model, baselines, statistics

IV-A Dataset and projects (Table II)

Project	Scenarios	Intents	Entity types	Pool	Test (in-scope + OOS)
P1 home assistant	alarm, iot	12	18	120	250 + 50
P2 productivity	calendar, email, lists	10	26	100	250 + 50
P3 ordering, travel (held out)	takeaway, transport	6	19	60	167 + 33

- Pool: 10 per intent from MASSIVE's training split. Dev: 80 in-scope from the official dev split + 20 OOS. Test: 250 from the official test split + 50 OOS.
- OOS requests come from 11 scenarios that belong to no project (audio, cooking, datetime, general, music, news, play, qa, recommendation, social, weather).
- P3's test is smaller (167 + 33) because MASSIVE's test split has only 181 takeaway/transport requests.
- The entity schema comes from the training split only, like a real developer who has not seen the test data. Side effect: 5 of P2's 26 entity types never appear in its pool (10.7% of its dev entities).
- P3 was fully locked by code (a data-access object that raises an error) until the freeze commit's hash was given.

IV-B Model and environment

Qwen2.5-3B-Instruct, 4-bit, run by Ollama 0.40.2 on a laptop with an NVIDIA RTX 3050 (4 GB). Temperature 0, seed 0, JSON output, at most 512 output tokens. 50 repeated requests gave 50 identical answers (deterministic). One call: 0.42 s median, 0.59 s p95 for prompts of about 1,028 tokens. Replies are cached by prompt fingerprint so results can be reproduced; cached answers are never used for timing.

IV-C Baselines: every system compared

Name	What it is	Why it is included
B0	TF-IDF word features + logistic regression, trained on the pool; refuses below a dev-tuned threshold; no entities.	The cheapest classic approach: shows how much the LM adds.
B1	Generic prompt: joint, fixed examples, model label, bare specification (no descriptions).	What a developer gets with zero design effort. The main reference for the success rule.
B2	Human designer: a team member read the dev results on a worksheet and picked one design (38 min on P1; under 6 min on P2 with a chat assistant, declared). Not run on P3.	Does the agent beat a person?
B3-k	XLM-R base fine-tuned on the same pool (30 epochs, learning rate 3×10^{-5}, batch 16); dev-tuned softmax threshold.	A trained model with the same data as the agent.
B3-full	Same, trained on the full training split (1,159 / 3,180 / 828 sentences, 5 epochs).	What training can do with 10–32× more data.
B4-random	4 random designs, same selection rule, 5 different random seeds.	The fair test of whether the agent's choices are smart.
B4-best	All 12 designs tested; the best on dev.	The best possible choice on dev (the ceiling).
A1 / A2	Our agents: direct planner / factor-wise planner.	The systems being tested.

Epoch = one pass over the training data. **Learning rate** = how big each training update is. **Batch 16** = the model learns from 16 sentences at a time.

IV-D Statistics and the success rule

Paired bootstrap [18]: imagine the 300 test requests as cards. Draw 300 cards with replacement (some twice, some not at all), score both systems on that same draw, note the difference. Repeat 1,000 times. The middle 95% of those differences is the 95% confidence interval. "Paired" means both systems are always scored on the same draw, which makes the comparison fair. The fixed seed (20261016) makes the draws repeatable.

The success rule (fixed before testing): compared with B1, the agent must (1) gain at least 2.0 points of intent F1, (2) lose at most 2.0 points of entity F1, (3) have schema validity at least 0.98, and (4) have p95 latency at most twice B1's measured in the same session.

Interleaved timing: 50 dev requests per system after 5 warm-up requests, sending each request to B1, A1 and A2 in turn, so if the laptop slows down it affects all three equally.

11. Section IV-E to IV-K: results, every table and figure

Table III: how to read it

Each cell has three numbers: **intent macro-F1 / entity span F1 / OOS F1**. Example: A2 on P1 = 0.898 / 0.699 / 0.865. A dash (–) means the system does not predict entities (B0). Bold = best language-model result in that column. B4-random shows the mean intent F1 of 5 seeds and the range (lowest–highest).

System	P1	P2	P3 (held out)	In one line
B0	0.699 / – / 0.426	0.669 / – / 0.403	0.769 / – / 0.404	Far behind; no entities; weak refusal.
B1	0.876 / 0.626 / 0.875	0.823 / 0.477 / 0.662	0.895 / 0.529 / 0.771	Generic prompt: strong start, best intent on P3.
B2	0.837 / 0.648 / 0.471	0.851 / 0.491 / 0.748	not run	Human chose a threshold on P1: refusal dropped to 0.471.
B3-k	0.809 / 0.650 / 0.568	0.800 / 0.461 / 0.536	0.770 / 0.515 / 0.505	Trained on the same small data: below A2 everywhere.
B3-full	0.883 / 0.802 / 0.604	0.904 / 0.775 / 0.560	0.913 / 0.721 / 0.581	Lots of data: best entities, weak refusal.
B4-random	0.893 (0.881–0.903)	0.854 (0.848–0.872)	0.894 (0.882–0.909)	Random 4 of 12.
A1	0.903 / 0.639 / 0.885	0.852 / 0.504 / 0.758	0.889 / 0.506 / 0.845	First agent.
A2	0.898 / 0.699 / 0.865	0.872 / 0.532 / 0.767	0.882 / 0.675 / 0.795	Improved agent: best entities of the LM systems.

Which designs were chosen: A2 picked schema-sim-llm on P1 and P2 and joint-sim-llm on P3. A1 picked twostage-sim-llm on P1 and schema-fixed-llm on P2 and P3. A2's choice equals B4-best (the best of all 12 on dev) on every project.

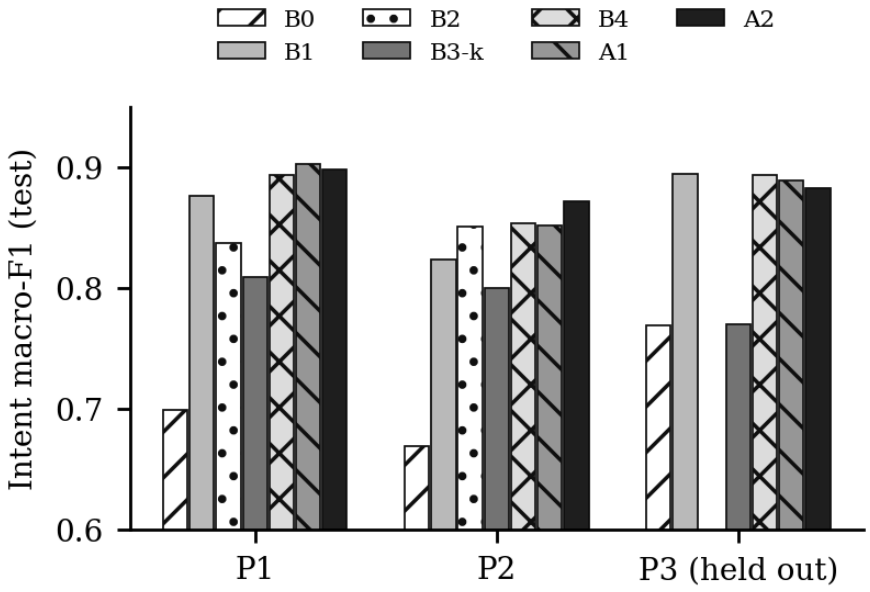


Figure 2 of the paper: test intent macro-F1 for each system and project. Each bar pattern is one system. The gap on P3 is B2, which was not run there.

The success rule result (paragraph "Success rule")

	P1	P2	P3
A2 vs B1 (intent / entity, points)	+2.2 / +7.3 → pass	+4.9 / +5.5 → pass	−1.3 / +14.6 → fail (intent)
A1 vs B1	+2.7 / +1.3 → pass	+2.9 / +2.7 → pass	−0.6 / −2.3 → fail (both)

Schema validity was at least 0.997 for every agent run (the rule needs 0.98). Latency p95: P1 A1 0.97 s and A2 0.89 s (limit 1.22 s); P2 A1 0.77 s and A2 1.08 s (limit 1.67 s). A2 is slower than B1 in the median (0.68 vs 0.45 s on P1) because similarity examples make longer prompts, but within the limit.

Significance and Figure 3

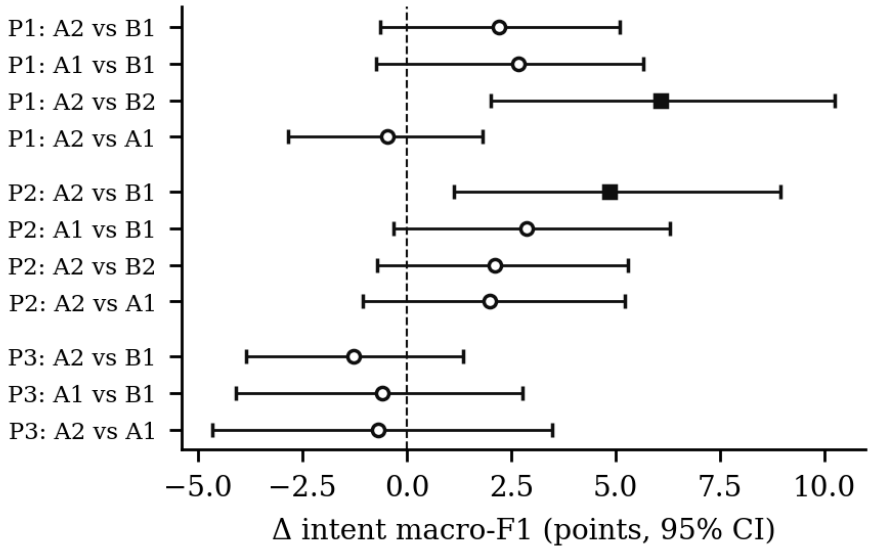


Figure 3 of the paper: each line is a 95% confidence interval of a difference in intent F1, in points. The dot is the measured difference. Filled square = the line does not cross the dashed zero line, so the difference is significant.

- A2 vs B1 intent: P2 +4.9 [+1.1, +8.9] significant; P1 +2.2 [−0.6, +5.1] not significant; P3 −1.3 [−3.8, +1.4] not significant.
- A2 vs B1 entity: significant on all three: P1 +7.3 [+0.8, +14.5], P2 +5.5 [+0.3, +11.1], P3 +14.6 [+9.0, +21.3].
- None of A1's differences from B1 is significant.
- Why P1 passes the rule but is not significant: with 200–300 test requests the interval is about ±3 points wide, so a gain just above 2 points can be real but cannot be proven. The paper says this openly.

IV-F Agent vs random search and vs the human

Random search averaged 0.893, 0.854 and 0.894. A2 was above the average on P1 and at the top of the range on P2, but at the bottom of the range on P3. A random seed has a 4 in 12 chance of including the best dev design, and then the same rule picks it. That happened in 1 of 5 seeds on P1 and P2 and 2 of 5 on P3. A2 got there every time without luck.

Against the human: A2 was 6.1 points higher on P1 (significant, [+2.0, +10.2]) and 2.1 higher on P2 (not significant). The human chose the threshold policy on P1, and refusal F1 fell to 0.471. On both dev projects all six model-label designs beat all six threshold designs, which is why the agent never kept a threshold design.

IV-G Agent vs fine-tuned XLM-R

Same small data (B3-k): A2 is 8.9, 7.2 and 11.2 points higher in intent F1, higher in entity F1, and far better at refusal (0.767–0.865 vs 0.505–0.568). With 10–32× more data (B3-full): XLM-R beats A2 on entities by 4.6–24.3 points and on intents for P2 and P3, but still refuses badly (below 0.61). Lesson: with few examples use the LM pipeline; with thousands of labelled sentences a trained encoder extracts entities better.

IV-H Why factor-wise planning worked (Table V)

Step	Design	P1 dev (intent / entity)	P2 dev
c1	schema, fixed, threshold	0.845 / 0.505	0.762 / 0.391
c2	schema, similarity, threshold	0.869 / 0.617	0.802 / 0.437
c3	two-stage, fixed, threshold	0.839 / 0.544	0.772 / 0.413
c4 (revision)	schema, similarity, model label	0.974 / 0.707	0.898 / 0.483

The story: the model preferred fixed examples and the threshold on both projects (on P1 it said the threshold saves model calls). Both preferences were wrong. But c2 is always built with the other example option, so similarity was tested and won. Then the revision, allowed only one-factor moves from c2, switched to the model label: +10.5 points on P1 and +9.6 on P2, reaching the best of all 12. A1 started from B1's design, never tried schema + similarity, and wasted its free revision on a threshold design (0.862 and 0.790). **The structure, not the model's guesses, made the difference.**

Honesty note in the paper: A2 was built after we studied A1's dev behaviour, so A2's dev scores on P1/P2 are not independent proof. Its test scores and all of P3 are, because everything was frozen before those were released.

IV-I Held-out project P3

A2 again picked the best dev design, but on test it was 1.3 points below B1. Reason: P3's dev set has only about 13 requests per intent, and the designs were very close on test (0.882–0.909), so the dev ranking did not carry over. Entities carried over strongly: +14.6 over B1 and +16.9 over A1, because similar examples show the model slot values like those in the request.

IV-J Ablations (Table VI)

Variant (part removed from A1)	P1	P2	P3	Meaning
A1 full	0.903	0.852	0.889	Reference.
without selection	0.876	0.823	0.895	Just use the first candidate (= B1's design): −2.7 and −2.9, the biggest effect.
without analysis enrichment	0.903	0.841	0.898	No descriptions: P2 intent −1.1, entity −7.4 (P2 has types missing from the pool).
without planner reasoning	0.903	0.848	0.882	Fixed fallback order instead of the model's reasoning.
without schema repair	0.907	0.852	0.889	Hardly matters: the model rarely broke the format.

IV-K Cost and reproducibility

- A2 design cost ≈ 397 model calls per project: 2 enrichment + 2 planning + ≈393 for testing 4 candidates on 100 requests (two-stage needs 2 calls per request, threshold refusals need 0). About a third of testing all 12, about 3 minutes of model time, against 38 minutes for the human on P1.
- Every metric was recomputed from saved predictions: 99 of 99 runs matched exactly. An independent library (scikit-learn) recomputed intent F1, accuracy and OOS precision/recall/F1 to 9 decimal places. B0 retrained from scratch gave identical test metrics.
- Extra check done after the paper was written (not in the paper): A2 was asked 50 P1 test requests live with the cache turned off, and all 50 answers were identical to the recorded ones (`reports/reproduction_live.json`). This proves the results come from the model itself, not from replaying saved answers.

12. Section V: Conclusion, acknowledgment and references

Conclusion: restates only what was done and found (teacher's rule: no limitations or future work there): the agent, the frozen protocol, best dev design on all three projects, success on P1 and P2, significant entity gains everywhere, the honest P3 result, beating XLM-R on the same data, and why it worked.

Acknowledgment: thanks the guide, states the MASSIVE licence (CC BY 4.0: free to use with credit) and declares AI assistance.

References: 18 entries in IEEE style, numbered in the order first cited. Each was checked against the publisher's page. GPT-3 has no page numbers because they could not be verified.

#	Paper	Used for
1	Chen et al. 2019, BERT joint intent and slot	Trained baseline design
2	Conneau et al. 2020, XLM-R	B3 baseline
3	FitzGerald et al. 2023, MASSIVE	Dataset
4	Zhou et al. 2023, APE	Prompt optimisation
5	Pryzant et al. 2023, ProTeGi	Prompt optimisation with error feedback
6	Yang et al. 2024, OPRO	LM as optimiser
7	Khattab et al. 2024, DSPy	Pipeline compilation
8	Viswanathan et al. 2023, Prompt2Model	Description → deployable model
9	Bergstra and Bengio 2012, random search	Why B4 is the fair control
10	Bastianelli et al. 2020, SLURP	Source of MASSIVE's English text
11	Larson et al. 2019, CLINC150	Out-of-scope detection
12	Brown et al. 2020, GPT-3	In-context learning
13	Liu et al. 2022, in-context examples	Similarity examples
14	Madaan et al. 2023, Self-Refine	Self-correction / repair
15	Yao et al. 2023, ReAct	Agent pattern
16	He et al. 2021, AutoML survey	AutoML framing
17	Qwen Team 2024, Qwen2.5	Our model
18	Koehn 2004, significance tests	Paired bootstrap

13. Where every number in the paper comes from

Every number can be traced to a file in the project folder `D:\MyClaudeProject\nlp`.

Number(s)	File
All test scores in Table III, Fig. 2, the success rule	`reports/final_test.json` (and B3 in `runs/*_B3-k-final_0`, `runs/*_B3-full-final_0`)
Confidence intervals, Fig. 3	`reports/final_test.json` → bootstrap
Latency (0.89 s, 1.22 s limit, medians)	`reports/timing_interleaved.json`
0.42 s / 0.59 s single call, 50/50 identical, model digest	`docs/protocol.md` section 1
Dev range 0.839–0.974 and 0.738–0.898	`runs/20261013_0112_P1_B4-exhaustive_0`, `runs/20261013_0115_P2_B4-exhaustive_0`
Table V (A2 dev trace)	`runs/20261025_1514_P1_A2_0/events.jsonl`, `runs/20261025_1514_P2_A2_0/events.jsonl`
A1 dev candidates (0.862, 0.790, 0.958, 0.841)	`runs/20261013_0110_P1_A1_0`, `runs/20261013_0112_P2_A1_0`
Table II, 1,159 / 3,180 / 828, 10.7%	`data/manifests/P1.json` … `P3.json`, `docs/decisions.md` (decision 34)
99 of 99 runs reproduced	`python scripts/reproduce_metrics.py`, `python scripts/verify_with_sklearn.py`
B0 retrained identically	`reports/reproduction_live.json`
38 min / under 6 min (B2)	B2 worksheets and `docs/decisions.md`
Figures	`python scripts/make_paper_figures.py` → `paper/figures/`

14. Questions a teacher or reviewer may ask, and the answers

Q: Is this just prompt engineering?

A: No. Prompt engineering is a person tuning one prompt by hand. We built an agent that chooses the whole pipeline structure automatically, tested it against a human and random search under a frozen protocol, and export a verified package.

Q: Why only 12 designs?

A: Each design must be safe, pre-written code (the agent never writes code), and each test costs about 100 model calls. Three factors that matter most for intent + slot tasks give 12. The 13–16 point spread shows they matter.

Q: Why only 4 tries?

A: To show the agent can find a good design cheaply. Testing all 12 costs three times as much; random search with 4 tries is the fair comparison.

Q: Why did it fail on P3?

A: On intents it was 1.3 points lower, which is within the confidence interval, so it may be luck. P3's dev set has only about 13 requests per intent, so the dev ranking did not hold on test. On entities it was 14.6 points better.

Q: Why does P1 pass the rule but is not significant?

A: The rule needs +2.0 points; we got +2.2. With 300 test requests the interval is about ±3 points, so the gain cannot be proven statistically. We report both honestly.

Q: Isn't A2 tuned on the dev data?

A: A2's method was designed after looking at A1's dev behaviour, and we say so. But it contains no hard-coded design, and its test results and the whole of P3 were locked until after the freeze, so those are fair evidence.

Q: Why not use a bigger model like GPT-4?

A: The point is a small model on a laptop: free, private and fast. All LM systems use the same model, so the comparison is fair.

Q: Why not just train XLM-R?

A: With the same 10 examples per intent, XLM-R is 7–11 points worse and refuses badly. It only wins on entities when given 10–32 times more labelled data, which a new app does not have.

Q: How do you know the numbers are right?

A: Every metric was recomputed from saved predictions (99 of 99 matched), checked independently with scikit-learn, and B0 retrained from scratch gave identical results. The model gives the same answer every time: asked live with the cache off, A2 reproduced 50 of 50 recorded test answers.

Q: What does the agent actually do with the language model?

A: Four calls per project: two to describe the labels, one to rank options per factor, one to choose the revision. Everything else (testing, scoring, selecting) is fixed code.

Q: What is new compared with DSPy or APE?

A: They tune instructions or examples inside a fixed structure, for a single output. We choose the structure, for intent + spans + rejection under constraints, and compare against random search.

Q: Which is your best system?

A: A2: best entity F1 among LM systems on all projects, best dev design every time, passes the rule on P1 and P2.